

Assignment 2 - Data Structures

Alua Rakhimzhanova | SE-2509 | Design and Analysis of Algorithms

Structures

DynamicArray and MinHeap use int arrays. The starting capacity is 4. When an array is full, its capacity doubles. MyLinkedList uses nodes with a value and a next link. It keeps head and tail pointers. DynamicArray and MyLinkedList have the same methods through IntList.

Benchmark

The sizes are 100, 1,000, 10,000 and 100,000. The data comes from new Random(42). Both lists get the same values and queries. Each case has 20 warm-up runs and 5 measured runs. The times are sorted, and the middle time is saved.

W1: 10,000 get calls at random indices.

W2: 1,000 searches. Half the queries are present. The other half are negative and absent.

W3: 1,000 insertions, then 1,000 removals. The index is 0 for head and the original $n / 2$ for middle.

W4: Insert n values into the heap, then extract all of them. Check that the output is sorted.

For W1-W3, filling the structure is done before timing. The counters are then reset. W4 measures both insertion and extraction. Generating data, checking results and saving CSV are outside the timer. The loops and counters are part of the measured time.

Counters

Steps: one array read or one move along a next link, including a move to null. Array get(i) takes 1 step. List get(i) takes i steps. Reading a node value does not add a step.

Moves: one existing array value moved, or one update to head, tail or next. Copying during growth also counts. A heap swap counts as 2 moves. A new array value, local variables and default null values do not count.

Comparisons: comparisons of values only. Index checks, loop conditions and null checks do not count. Checks in tests do not change the benchmark counters.

Tests

All 18 JUnit tests passed. They check random operations, empty structures, duplicates, invalid indices, growth and counters. Heap tests also check sorted output and parent $\leq$ child after changes.

The results were measured on Windows 11 with an AMD Ryzen 5 5500U and Java 25.0.1. The project uses Java 17 source level.

Complexity of the operations

n is the number of stored values. C is the array capacity. For average indexed operations, each valid index has the same chance. For search, a match can be at any position, and some queries are absent. The table includes the cost of copying when an array grows.

Operation	Best	Average	Worst	Auxiliary	Reason
Array add(x)	$\Theta(1)$	$\Theta(1)$ space; $\Theta(n)$ full*	$\Theta(n)$	$\Theta(n)$ growth; $\Theta(1)$ else	Normally one write; growth copies n values.
Array add(i,x)	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$ growth; $\Theta(1)$ else	Shifts $n-i$ values; may also grow.
Array remove(i)	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Shifts $n-i-1$ values; last removal is constant.
Array get(i)	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	Reads <code>data[i]</code> directly.
Array contains(x)	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Checks values until a match or the end.
List add(x)	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	Uses tail; allocates one node.
List add(i,x)	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Head/tail are direct; otherwise find predecessor.
List remove(i)	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Head is direct; otherwise find predecessor.
List get(i)	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Follows i next links from head.
List contains(x)	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Visits nodes until a match or null.
Heap insert(x)	$\Theta(1)$	$O(\log n)$ space; $\Theta(n)$ full*	$\Theta(n)$	$\Theta(n)$ growth; $\Theta(1)$ else	Bubble-up has at most $\log n$ levels; growth copies n values.
Heap peekMin()	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	Reads the root.
Heap extractMin()	$\Theta(1)$	$\Theta(\log n)$ **	$\Theta(\log n)$	$\Theta(1)$	Bubble-down stops early or reaches the bottom.

* 'Space' means there is a free slot. 'Full' means $n = C$. A full array must copy n values. With a free slot, heap insertion takes between $\Omega(1)$ and $O(\log n)$, depending on the values. $O(\log n)$ is an upper bound here. One array append can take $\Theta(n)$, but a sequence of appends costs $\Theta(1)$ per append on average over the sequence. This is the amortized cost. Heap insertion is $O(\log n)$ amortized.

** Heap extraction has expected $\Theta(\log n)$ cost for distinct random values. If all values are equal, it can stop early in $\Theta(1)$. Searching an empty structure and rejecting an invalid operation also take $\Theta(1)$.

`size()`, `metrics()` and `reset()` take $\Theta(1)$ time and space. The heap snapshot used in tests copies C cells, so it takes $\Theta(C)$ time and extra space. Total storage is $\Theta(C)$ for arrays and $\Theta(n)$ for the list. Arrays do not shrink after removals.

Loop invariants and discussion

1. `DynamicArray.contains(value)`

Invariant: Before iteration i , $0 \leq i \leq \text{size}$. None of the values before i equals `value`. The array and size stay the same.

Initialization: $i = 0$. No values have been checked, so the statement is true.

Maintenance: If `data[i]` matches, return true. Otherwise, it is also not a match. After i increases, every value before i is still not a match.

Termination: A match gives true. If i reaches `size`, all values were checked and none matched, so return false.

Conclusion: The method returns true exactly when the value is in the array.

2. `DynamicArray.remove(index)`

A is the original array, n is its size, and k is the removal index. $A[k]$ is saved before the loop.

Invariant: Before iteration i , $k \leq i \leq n-1$. Values before k are unchanged. For $k \leq j < i$, `data[j] = A[j+1]`. For $i \leq j < n$, `data[j] = A[j]`. This means the part before i is already shifted, and the rest is unchanged.

Initialization: $i = k$. Nothing has been shifted yet.

Maintenance: `data[i+1]` still holds $A[i+1]$. Copying it to `data[i]` shifts one more value left. Then i increases by 1.

Termination: $i = n-1$. All values after k have moved left. Decreasing size removes the unused last slot.

Conclusion: The array keeps its order with $A[k]$ removed. The method returns the saved value. For the last index, no shifting is needed.

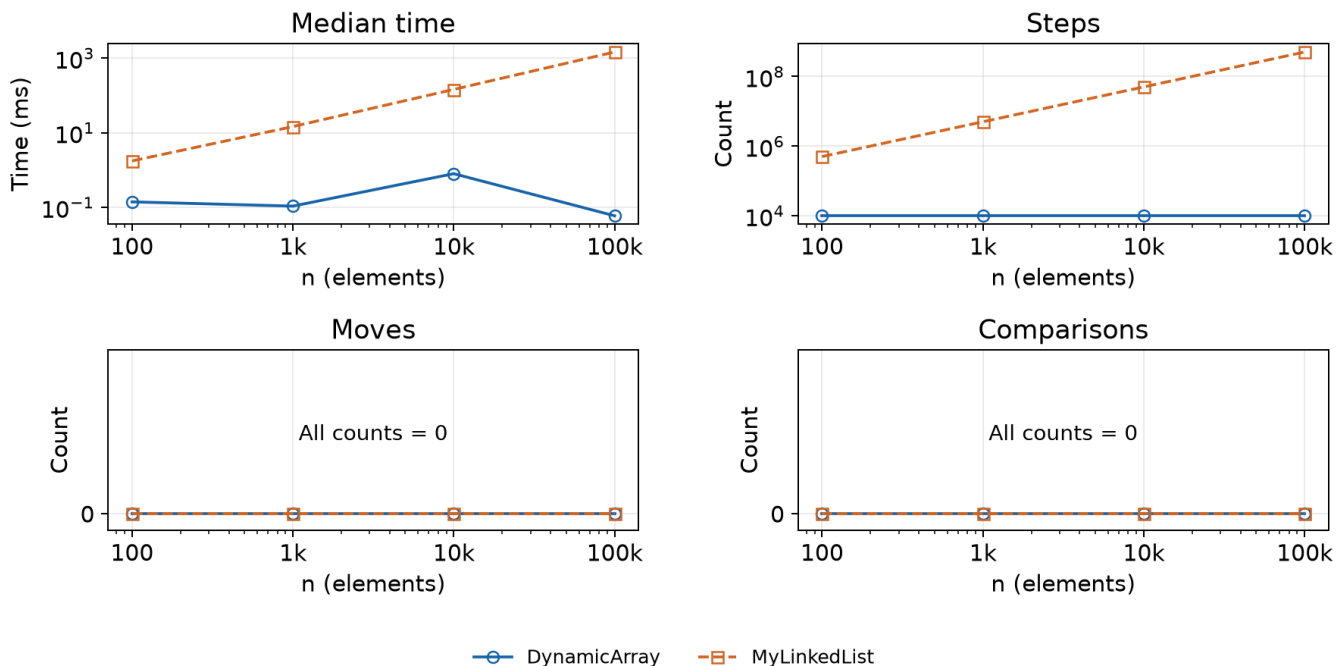
3. Discussion

`DynamicArray` gets a value directly by index. `MyLinkedList` must follow links from the head. Array values are next to each other in memory, which is called spatial locality. One CPU cache line can load several nearby values, so scanning an array is also fast. List nodes may be far apart in memory. The program must read one node to find the next one, which is called pointer chasing. This is why the same Big-O or similar step counts can still give different times. Each node also needs an object header, a link and some padding for alignment. Creating and removing nodes adds work for the garbage collector, but this test does not measure that work separately. In W2 at $n = 100,000$, both structures made 73,682,044 comparisons, but the times were 86.316 ms for the array and 273.177 ms for the list. `MyLinkedList` is useful for adding and removing values at the head. Middle operations still need a walk through the list, and removing the tail also needs its previous node. `MinHeap` is useful for jobs where the smallest priority must be handled first. It finds the minimum in $\Theta(1)$ and removes it in $O(\log n)$. Short timings can vary because of Java compilation and other work on the computer.

Measurements: access and search

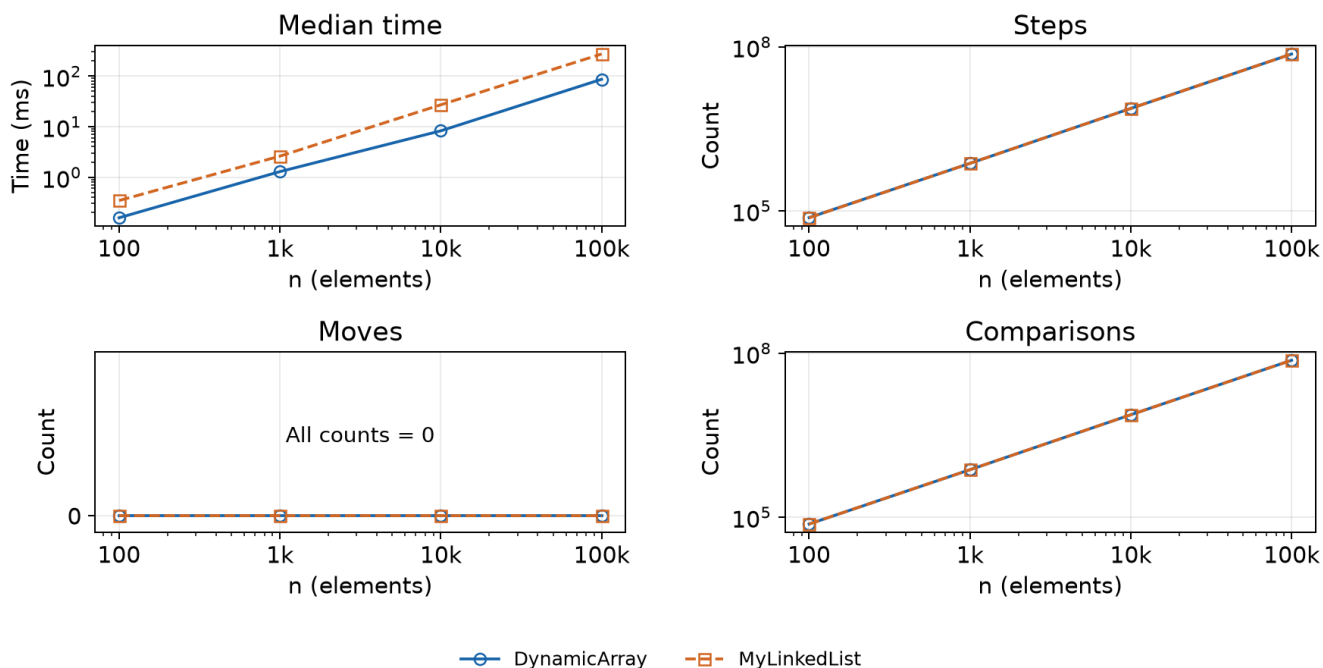
The graphs show median time and operation counts. Nonzero graphs use log scales. Zero counts use a normal scale. Equal lines may overlap.

W1 - Random access



W1: the array always makes 10,000 reads. The list needs more steps as n grows. At n = 100,000, the array takes 0.060 ms and the list takes 1,451.316 ms.

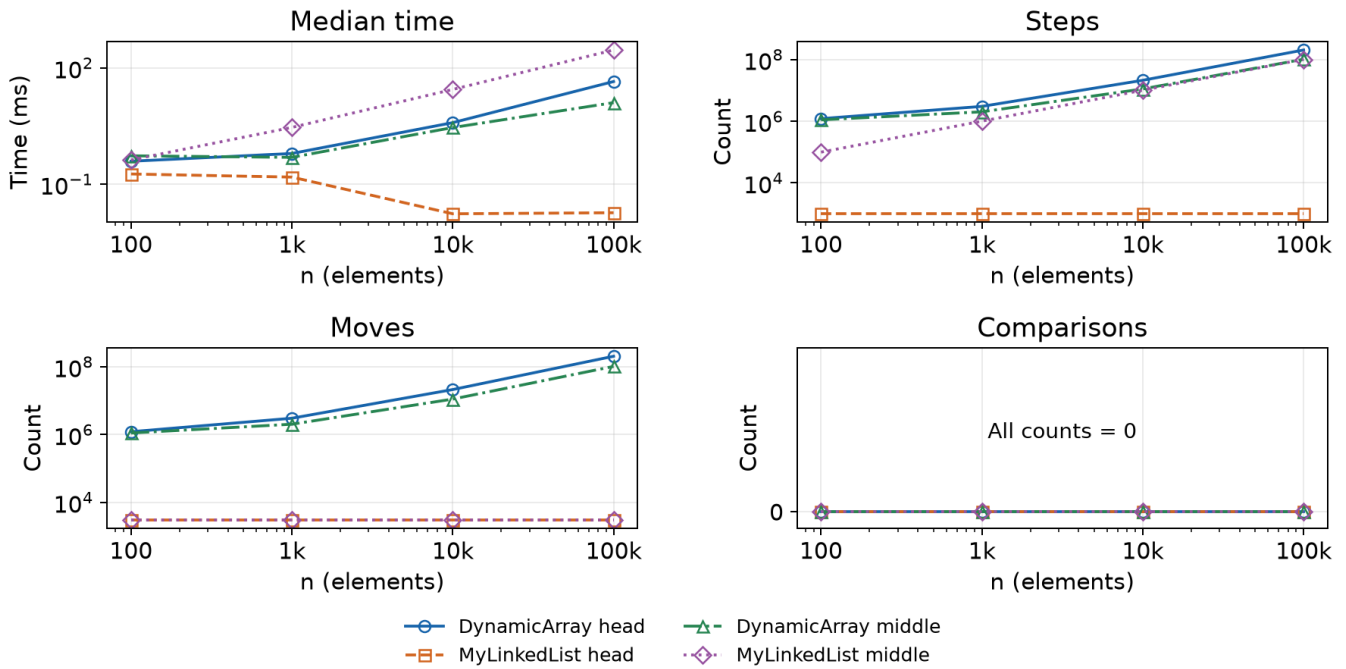
W2 - Search



W2: both structures check the same values, so they make the same number of comparisons. The list has 500 fewer steps. Each successful search stops at the matching node without following another link. No values move.

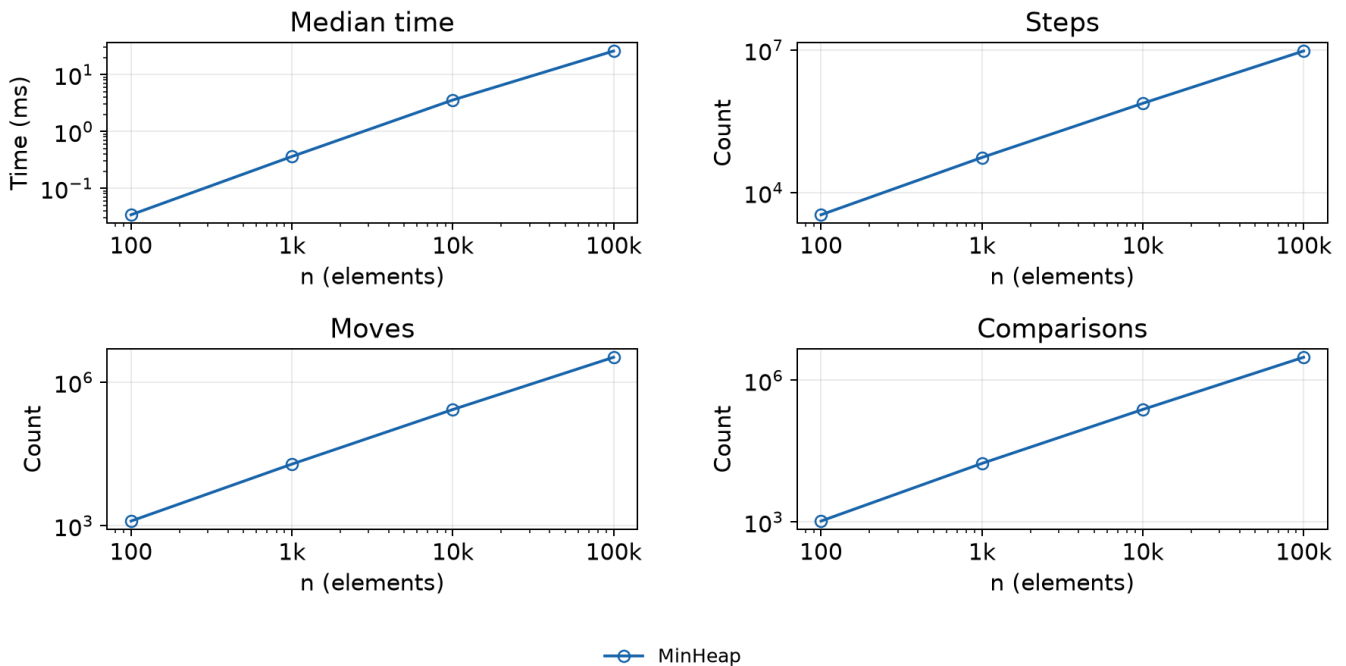
Measurements: updates and priority

W3 - Insert and remove



W3: at $n = 100,000$, head operations take 44.101 ms for the array and 0.018 ms for the list. Middle operations take 12.604 ms and 286.180 ms. Both list cases make 3,000 link updates. The middle case also needs many steps to reach the index. Array counts include growth copies. There are no value comparisons.

W4 - Priority processing



W4: inserting and extracting 100,000 values takes 25.882 ms. The heap makes 9,505,677 steps, 3,287,427 moves and 3,059,125 comparisons. Total time is $O(n \log n)$. All returned values were in sorted order.